

Phase 1 Report

1. Overview

This project is a source-level obfuscator for a simplified version of the C programming language (Mini-C).

It supports basic types, expressions, control structures (if/else, while, for), I/O with printf/scanf, and function declarations/calls.

The goal of the obfuscator is to transform readable code into functionally equivalent but harder-to-read code using:

- Variable renaming
- Expression obfuscation
- Dead code insertion

2. Implemented Obfuscation Techniques

- **Identifier Renaming:**

Variables and function names (except main) are renamed to meaningless short names (like a0, a1, etc.).

- **Expression Obfuscation:**

Every constant or identifier in an expression is replaced with an equivalent, more complex form.

Example: $x + y \rightarrow (x * 1 + (y + 0))$

- **Dead Code Insertion:**

Random dead assignments or conditions like `if (0) {...}` are inserted in the code.

3. Code Structure

The project consists of five main classes:

- `Lexer.java`: Scans the input and produces a list of tokens.

- Parser.java: Parses the token stream and writes the obfuscated version to a file.
- Main.java: Loads input/output files and triggers the obfuscation pipeline.
- Token.java: Represents a token with type and value.
- TokenType.java: Enum of token types (e.g., KEYWORD, IDENTIFIER, NUMBER, etc.).

4. Explanation of Key Classes

◆ Lexer Class – Tokenizer of Mini C

The Lexer is responsible for **converting the raw Mini C code into tokens** that can later be parsed by the Parser. It scans the code character by character and generates a list of Token objects with their type and value.

Key Responsibilities of the Lexer:

1. Ignore whitespaces and comments

It skips over spaces, newlines, and supports ignoring C-style comments (`// ...`).

2. Recognize keywords

It identifies reserved words like:

- int, char, void, if, else, while, for, return
- printf, scanf, main

3. Detect identifiers

Any variable/function name starting with a letter or `_` and followed by letters, digits, or underscores is classified as an IDENTIFIER.

4. Handle numbers

Both integers and floating-point numbers are tokenized as NUMBER.

5. Handle operators and symbols

Including:

- Arithmetic: +, -, *, /
- Relational: ==, !=, <, >, <=, >=
- Assignment: =
- Symbols: (,), {, }, :, ,, &

6. Support string and char literals

Detects quoted literals like "%d" and 'a'.

Token Output:

Each recognized piece becomes a Token with a type (like KEYWORD, IDENTIFIER, NUMBER, etc.) and its actual value. These tokens are stored in a list and later passed to the Parser.

◆ Parser Class – Mini C Syntax Analyzer & Obfuscator

The Parser is responsible for analyzing the token stream produced by the Lexer, checking for syntactic correctness, and generating a functionally equivalent but obfuscated version of the original Mini C code. It performs **recursive descent parsing**, following the grammar rules of Mini C, and simultaneously applies multiple obfuscation techniques.

Key Responsibilities of the Parser:

1. Parse and Obfuscate Statements

The parser handles different kinds of statements:

- Variable declarations and assignments
- Control flow structures (if, while, for)
- Function declarations and function calls
- Input/output (printf, scanf)
- Return statements

2. Apply Identifier Obfuscation

All variable and function names (except main) are replaced with meaningless names like a0, a1, a2,

etc., using a consistent mapping to preserve behavior.

3. Expression Transformation

Arithmetic expressions are parsed recursively, and constants or identifiers inside them are replaced with equivalent but more complex expressions.

Example: $x + 1$ might be transformed into $(x * 1 + (1 + 0))$

4. Dead Code Insertion

Randomly inserts fake code such as:

- Useless assignments: `int dummy = 5 - 5;`
- Dead conditions: `if (0) { int temp = 0; }`

These have no effect on functionality but increase complexity.

5. Maintain Output via File Writer

The obfuscated version of the code is printed both to the console and to a .mc output file using a shared PrintWriter.

6. Handle Nesting and Blocks

Supports nested if, while, and for structures with proper `{}` scoping and parsing of statements inside blocks.

7. Special Case Handling

- `main` is never obfuscated.
- Function calls can be used inline (as `return` values or expressions).
- Proper detection and parsing of negative numbers, compound expressions, and multi-argument functions.

The parser acts as both the **syntactic engine** and **obfuscation core** of the Mini C translator. It ensures the final transformed code behaves exactly like the original, while appearing much harder to read or reverse-engineer.

Key Functions:

◆ `parseStatement()`

Parses an if-else if-else block. It checks the condition, then recursively processes the inner statements. It supports nesting and complex branching logic. All scopes are handled using {} blocks, and both console and file outputs are generated.

◆ `parseForLoop()`

Parses a for loop. It handles:

- Initialization (like `int i = 0`)
- Loop condition (like `i < n`)
- Increment statement (like `i = i + 1`)

The loop is then printed as a full for (init; condition; increment) with obfuscated variable names and transformed expressions.

◆ `parseWhileLoop()`

Parses a while loop. It extracts a condition like `x < 10`, prints the start of the loop, and then parses the inner block by recursively calling `parseStatement()` until it reaches `}`.

◆ `parseFunctionDeclaration()`

Parses function definitions such as `int add(int a, int b) { ... }`. It obfuscates:

- The function name (unless it's main)
- All parameter names

Then processes the function body recursively.

◆ `parseReturnStatement()`

Parses a return statement. It reads and obfuscates the return expression, then prints it with a ;.

◆ `parseAssignment()`

Parses assignments like `x = 5 + y;`. It obfuscates the variable name on the left and transforms the expression on the right using the expression parser.

◆ `parseVariableDeclaration()`

Parses variable declarations like `int x = 5;` or `char c;`. The variable name is obfuscated, and the initialization expression (if present) is parsed and transformed.

◆ `parseExpression()`, `parseTerm()`, `parseFactor()`, `parsePrimary()`

This family of functions performs recursive descent parsing of mathematical expressions:

- `parseExpression()` handles + and -
- `parseFactor()` handles * and /
- `parsePrimary()` handles literals, identifiers, parentheses, and negative numbers

It also applies expression-level obfuscation like:

- $x + y \rightarrow x - (-y)$
- $a \rightarrow a + 0$, $b \rightarrow b * 1$, etc.

◆ **parsePrintf()**

Parses printf statements like `printf("%d", x);`. The format string is preserved (with quotes), and any variables are obfuscated properly.

◆ **parseScanf()**

Parses scanf statements like `scanf("%d", &x);`. The format string is preserved, and the variable is obfuscated while maintaining the & syntax.

◆ **maybeInsertDeadCode()**

Randomly injects “dead code” (code that does nothing), like:

- `int dummy = 0;`
- `if (0) { int dummy = 5; }`

These blocks don't affect the program but help obfuscate the logic.

◆ **getObfuscatedName(String name)**

Generates and returns an obfuscated version of a variable/function name using a mapping like:

- `x` → `a0`, `y` → `a1`, etc.

This ensures consistent and reversible name obfuscation.

◆ **obfuscateIdentifier(String name), obfuscateNumber(String value)**

These two functions wrap variables and numbers inside harmless but confusing transformations like:

- `x` → `(x + 0)` or `(x * 1)`

- $5 \rightarrow (10 / 2) \text{ or } (5 + 0)$

5. Design Decisions

- Recursive-descent parser for complete control
- Manual handling of nested blocks with `{}`
- Prevent renaming main
- Inserted randomness in obfuscation for variation
- Expressions parsed recursively to allow nesting

6. Before and After Example

input:

```

3- int average(int a, int b, int c) {
4     int sum = a + b + c;
5     int avg = sum / 3;
6     return avg;
7 }
8
9- int main() {
10     int x = 0;
11     int y = 0;
12     int z = 0;
13     scanf("%d", &x);
14     scanf("%d", &y);
15     scanf("%d", &z);
16
17     int avg = average(x, y, z);
18
19-     if (avg > 50) {
20         printf("Passed\n");
21     } else {
22         if (avg == 50) {
23             printf("Barely passed\n");
24         } else {
25             printf("Failed\n");
26         }
27     }
28
29     return 0;
30 }
31

```

Output:

```

2  #include <stdio.h>
3  int a0(int a1, int a2, int a3) {
4  int deadVar24 = 24 - 24;
5  int a4 = (((a1 - 0) + (a2 / 1)) + (a3 * 1));
6  int a5 = ((a4 + (a4 - a4)) / (6.0 / 2));
7  return (a5 * 1);
8  if (1) { int tmp = 0; tmp = tmp; }
9  }
10 int main() {
11 if (0) { int x = 29; }
12 int a6 = (10.0 - 10);
13 int a7 = (0.0 / 2);
14 int a8 = (0.0 / 2);
15 scanf("%d", &a6);
16 scanf("%d", &a7);
17 scanf("%d", &a8);
18 int a5 = a0((a6 * 1), (a7 - 0), (a8 + (a8 - a8)));
19 if (a5>50) {
20 printf("Passed\n");
21 }
22 else {
23 if (a5==50) {
24 printf("Barely passed\n");
25 }
26 else {
27 printf("Failed\n");
28 }
29 }
30 return (0 + 0);
31 int deadVar94 = 94 - 94;
32 }
33
34

```

7. Challenges

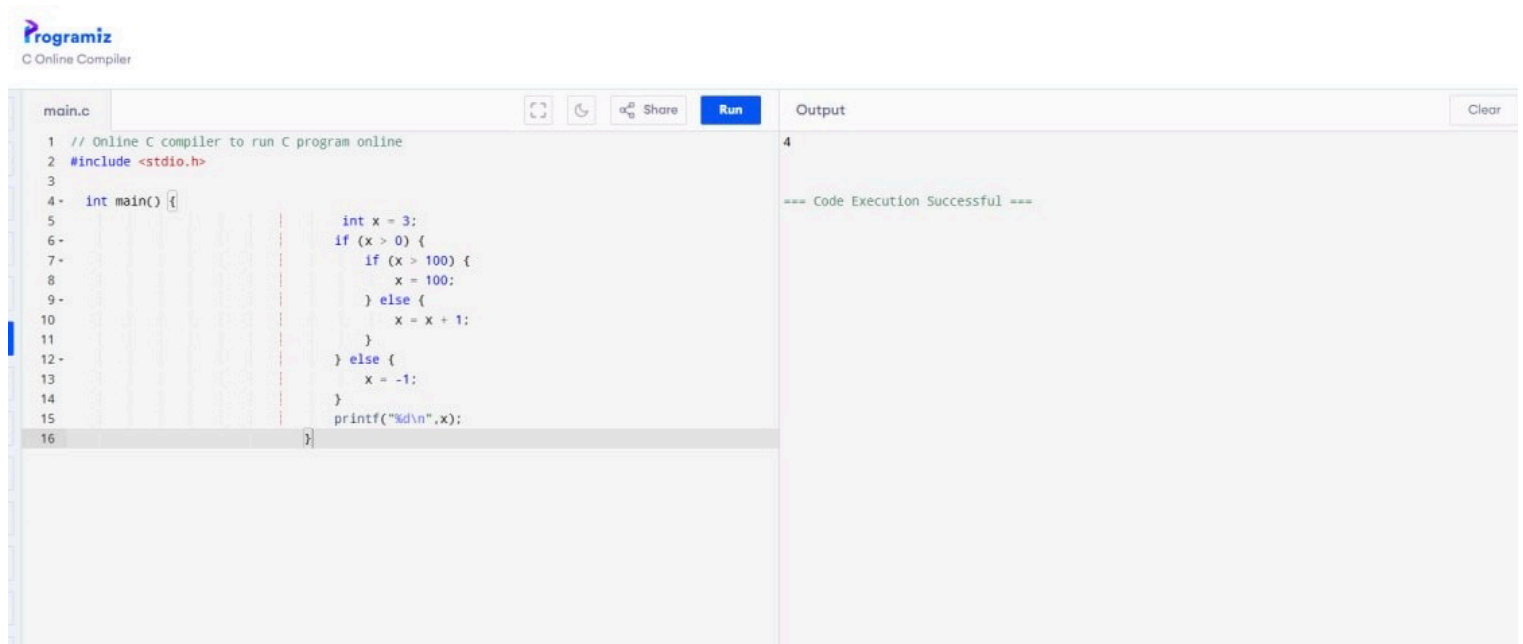
- Supporting nested structures and mixed syntax
- Ensuring expressions remain semantically valid
- Avoiding over-obfuscation that could break formatting
- Handling function calls in conditions/returns robustly

8. Equivalence Testing

The obfuscated code is tested against the original using sample inputs to ensure **identical behavior**.

Screenshots and outputs are included separately as proof.

Input1:



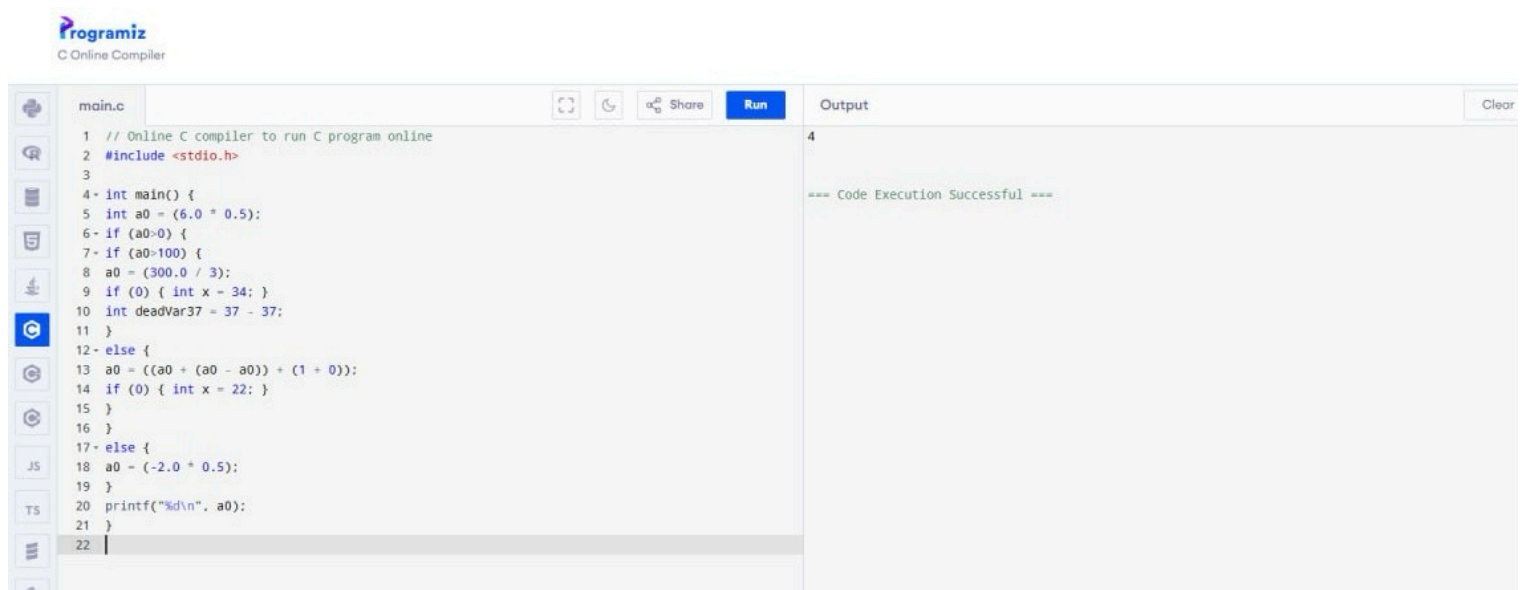
The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program named 'main.c'. The code is as follows:

```
1 // Online C compiler to run C program online
2 #include <stdio.h>
3
4 int main() {
5     int x = 3;
6     if (x > 0) {
7         if (x > 100) {
8             x = 100;
9         } else {
10            x = x + 1;
11        }
12    } else {
13        x = -1;
14    }
15    printf("%d\n", x);
16 }
```

The 'Run' button is highlighted in blue. The output panel on the right shows the result of the execution:

```
4
=== Code Execution Successful ===
```

Output1:



The screenshot shows the Programiz C Online Compiler interface. The code editor on the left contains a C program named 'main.c'. The code is as follows:

```
1 // Online C compiler to run C program online
2 #include <stdio.h>
3
4 int main() {
5     int a0 = (6.0 * 0.5);
6     if (a0 > 0) {
7         if (a0 > 100) {
8             a0 = (300.0 / 3);
9             if (0) { int x = 34; }
10            int deadVar37 = 37 - 37;
11        }
12    } else {
13        a0 = ((a0 + (a0 - a0)) + (1 + 0));
14        if (0) { int x = 22; }
15    }
16 }
17 else {
18     a0 = (-2.0 * 0.5);
19 }
20 printf("%d\n", a0);
21 }
22
```

The 'Run' button is highlighted in blue. The output panel on the right shows the result of the execution:

```
4
=== Code Execution Successful ===
```

Input2:

Programiz
 C Online Compiler

main.c

Run

Output
 Clear

```

1 // Online C compiler to run C program online
2 #include <stdio.h>
3
4 int a0(int a1, int a2) {
5     if (0) { int x = 28; }
6     int a3 = ((a1 + (a1 - a1)) + (a2 * 1));
7     return (a3 / 1);
8     if (0) { int x = 29; }
9 }
10
11 int main() {
12     int deadVar11 = 11 - 11;
13     int a4 = (9.0 / 3);
14     int a5 = (12.0 / 3);
15     int a6 = a0((a4 / 1), (a5 + 0));
16     printf("%d\n", a6);
17     if (0) { int x = 94; }
18 }
19
20

```

7

=== Code Execution Successful ===

Input3:

Programiz
C Online Compiler

main.c

1 // Online C compiler to run C program online

2 #include <stdio.h>

3

4 int compute(int a) {

5 if (a == 0) {

6 return 0;

7 } else {

8 return a * 2;

9 }

10 }

11 int main() {

12 int i;

13 int val = 0;

14 scanf("%d", &val);

15 i = compute(val);

16 printf("%d", i);

17

18 return 0;

19 }

Output

5

10

=== Code Execution Successful ===

Output3:

Programiz
C Online Compiler

main.c

1 // Online C compiler to run C program online

2 #include <stdio.h>

3

4 int a0(int a1) {

5 int deadVar32 = 32 - 32;

6 if (a1--0) {

7 return (0 + 0);

8 }

9 else {

10 return ((a1 - 0) * (2 + 0));

11 }

12 if (0) { int x = 70; }

13 }

14 int main() {

15 for (int i = 0; i < 0; i = i + 1) { int z = 0; }

16 int a2;

17 int a3 = (0 + 0);

18 scanf("%d", &a3);

19 a2 = a0((a3 / 1));

20 for (int i = 0; i < 0; i = i + 1) { int z = 0; }

21 if (0) { int x = 81; }

22 printf("%d", a2);

23 return (10.0 - 10);

24 int deadVar56 = 56 - 56;

25 }

26

27

28

29

Output

5

10

=== Code Execution Successful ===

Input4:

Programiz

C Online Compiler

main.c

Share

Run

Output

Clear

```
1 // Online C compiler to run C program online
2 #include <stdio.h>
3 int a0(int a1, int a2, int a3) {
4     int deadVar24 = 24 - 24;
5     int a4 = (((a1 - 0) + (a2 / 1)) + (a3 * 1));
6     int a5 = (((a4 + (a4 - a4)) / (6.0 / 2));
7     return (a5 * 1);
8     if (1) { int tmp = 0; tmp = tmp; }
9 }
10 int main() {
11     if (0) { int x = 29; }
12     int a6 = (10.0 - 10);
13     int a7 = (0.0 / 2);
14     int a8 = (0.0 / 2);
15     scanf("%d", &a6);
16     scanf("%d", &a7);
17     scanf("%d", &a8);
18     int a5 = a0((a6 * 1), (a7 - 0), (a8 + (a8 - a8)));
19     if (a5>50) {
20         printf("Passed\n");
21     }
22     else {
23         if (a5==50) {
24             printf("Barely passed\n");
25         }
26         else {
27             printf("Failed\n");
28         }
29     }
30     return (0 + 0);
31     int deadVar94 = 94 - 94;
32 }
33
34
```

49 56 678

Passed

=== Code Execution Successful ===

Input5:

Programiz

C Online Compiler

main.c

1 // Online C compiler to run C program online

2 #include <stdio.h>

3

4 int isEven(int n) {

5 while (n >= 2) {

6 n = n - 2;

7 }

8 if (n == 0) {

9 return 1;

10 } else {

11 return 0;

12 }

13 }

14

15 int main() {

16 int sum = 0;

17 int limit = 0;

18 int Temp = 0;

19 scanf("%d", &limit);

20

21 for (int i = 1; i <= limit; i = i + 1) {

22 Temp = isEven(i);

23 if (Temp == 1) {

24 sum = sum + i;

25 }

26 }

27

28 printf("Sum of evens: %d\n", sum);

29 return 0;

30 }

Output

14

Sum of evens: 56

=== Code Execution Successful ===

Output5:

Programiz

C Online Compiler

main.c

1 // Online C compiler to run C program online

2 #include <stdio.h>

3 int a0(int a1) {

4 int deadVar43 = 43 - 43;

5 while (a1 >= 2) {

6 a1 = ((a1 * 1) - (12.0 - 10));

7 if (1) { int tmp = 0; tmp = tmp; }

8 for (int i = 0; i < 0; i = i + 1) { int z = 0; }

9 }

10 if (a1==0) {

11 return (2.0 * 0.5);

12 }

13 else {

14 return (0.0 / 3);

15 }

16 }

17 int main() {

18 for (int i = 0; i < 0; i = i + 1) { int z = 0; }

19 int a2 = (0.0 / 3);

20 int a3 = (0.0 / 2);

21 int a4 = (0.0 * 0.5);

22 scanf("%d", &a3);

23 for (int a5 = (3.0 / 3); a5 <= a3; a5 = ((a5 / 1) + (11.0 - 10))) {

24 a4 = a0((a5 / 1));

25 if (0) { int x = 66; }

26 if (0) { int x = 86; }

27 if (a4--1) {

28 a2 = ((a2 - 0) + (a5 - 0));

29 if (1) { int tmp = 0; tmp = tmp; }

30 int deadVar48 = 48 - 48;

31 }

32 }

33 printf("Sum of evens: %d\n", a2);

34 return (10.0 - 10);

35 int deadVar96 = 96 - 96;

36 }

Output

14

Sum of evens: 56

=== Code Execution Successful ===